Phase 1 開発環境セットアップ完了

作成日: 2025年10月7日
ステータス: ✅ 完了

概要

Phase 1 の開発環境セットアップが完了しました。Docker を使用した統一された開発環境を構築しました。

完了内容

1. Docker 環境構築

サービス	ポート	説明	ヘルスチェック
postgres	5432	PostgreSQL 15	pg_isready
redis	6379	Redis 7	redis-cli ping
backend	8000	FastAPI (Python 3.11)	http://localhost:8000/health
frontend	3000	Next.js (Node 20)	http://localhost:3000

2. 作成ファイル

Docker 関連:
- `docker-compose.yaml` - 全サービス定義
- `backend/Dockerfile` - Backend コンテナ定義
- `frontend/Dockerfile` - Frontend コンテナ定義
- `.dockerignore` - ビルド除外設定

Backend 設定:
- `backend/pyproject.toml` - Poetry 依存関係
- `backend/alembic.ini` - Alembic 設定
- `backend/alembic/env.py` - マイグレーション環境
- `backend/app/main.py` - FastAPI アプリケーション
- `backend/app/core/config.py` - 設定管理
- `backend/app/core/database.py` - データベース接続
- `backend/app/core/logging.py` - ログ設定
- `backend/app/models/base.py` - ベースモデル
- `backend/app/models/mixins.py` - ミックスイン
- `backend/app/models/__init__.py` - モデルエクスポート
- `backend/app/api/v1/__init__.py` - API ルーター
- `backend/app/api/v1/health.py` - ヘルスチェック API
- `backend/app/middleware/__init__.py` - ミドルウェア
- `backend/app/middleware/request_id.py` - リクエスト ID ミドルウェア
- `backend/app/middleware/error_handler.py` - エラーハンドラー
- `backend/app/schemas/__init__.py` - スキーマ
- `backend/app/schemas/common.py` - 共通スキーマ
- `backend/app/services/__init__.py` - サービス
- `backend/app/utils/__init__.py` - ユーティリティ
- `backend/app/utils/exceptions.py` - 例外定義
- `backend/app/utils/response.py` - レスポンス形式
- `backend/tests/__init__.py` - テスト
- `backend/tests/conftest.py` - pytest 設定
- `backend/tests/test_health.py` - ヘルスチェックテスト

Frontend 設定:
- `frontend/package.json` - 依存関係
- `frontend/tsconfig.json` - TypeScript 設定
- `frontend/next.config.mjs` - Next.js 設定
- `frontend/tailwind.config.ts` - Tailwind CSS 設定
- `frontend/postcss.config.mjs` - PostCSS 設定
- `frontend/app/layout.tsx` - ルートレイアウト
- `frontend/app/page.tsx` - トップページ
- `frontend/app/globals.css` - グローバル CSS
- `frontend/app/api/health/route.ts` - ヘルスチェック API
- `frontend/lib/utils.ts` - ユーティリティ
- `frontend/components.json` - shadcn/ui 設定

3. 起動確認

```
$ docker compose ps
NAME                   STATUS                    PORTS
backend                Up (healthy)              0.0.0.0:8000->8000/tcp
frontend               Up (health: starting)     0.0.0.0:3000->3000/tcp
postgres               Up (healthy)              0.0.0.0:5432->5432/tcp
redis                  Up (healthy)              0.0.0.0:6379->6379/tcp
```

```
$ curl http://localhost:8000/health
{"status":"ok"}
```

```
$ curl http://localhost:3000/api/health
{"status":"ok"}
```

次のステップ

1. データベースモデル

参考資料

- Phase 1 タスク一覧
- 開発環境セットアップ手順

関連ファイル

- `docker-compose.yaml`
- `backend/app/main.py`
- `frontend/app/page.tsx`

データベースモデル

作成日: 2025年10月7日
ステータス: ✅ 完了

概要

データベースモデルの実装が完了しました。SQLAlchemy 2.0 を使用した型安全なモデル定義を実装しました。

完了内容

1. ER 図

2. 実装内容

ベースモデル:
- `UuidPkMixin` - UUIDv7 主キー
- `TimestampMixin` - created_at / updated_at
- `SoftDeleteMixin` - 論理削除
- `AuditMixin` - 監査ログ
- `BaseModel` - 基本モデル

User モデル:
- `id` - UUIDv7 主キー
- `email` - ユニーク、インデックス
- `hashed_password` - bcrypt ハッシュ
- `is_active` - アクティブフラグ
- `is_superuser` - 管理者フラグ
- `created_at` / `updated_at` - タイムスタンプ
- `owned_workspaces` - 所有ワークスペース
- `workspace_memberships` - ワークスペースメンバーシップ
- `assigned_tasks` - 割り当てタスク

Workspace モデル:
- `id` - UUIDv7 主キー
- `name` - ワークスペース名
- `slug` - URL 用識別子（ユニーク）
- `owner_id` - 所有者 ID
- `owner` - 所有者
- `members` - メンバー
- `projects` - プロジェクト

WorkspaceMember モデル:
- `user_id` - ユーザー ID（複合主キー）
- `workspace_id` - ワークスペース ID（複合主キー）
- `role` - ロール（owner / admin / member / guest）
- `joined_at` - 参加日時
- `user` - ユーザー
- `workspace` - ワークスペース

Project モデル:
- `id` - UUIDv7 主キー
- `workspace_id` - ワークスペース ID
- `name` - プロジェクト名
- `description` - 説明
- `workspace` - ワークスペース
- `tasks` - タスク

Task モデル:
- `id` - UUIDv7 主キー
- `project_id` - プロジェクト ID
- `title` - タイトル
- `description` - 説明
- `status` - ステータス（todo / in_progress / done / cancelled）
- `priority` - 優先度（low / medium / high / urgent）
- `assignee_id` - 担当者 ID
- `due_date` - 期限
- `project` - プロジェクト
- `assignee` - 担当者

3. マイグレーション

作成ファイル:
- `alembic/versions/0001_initial.py` - 初期マイグレーション

テーブル:
- `users` - ユーザー
- `workspaces` - ワークスペース
- `workspace_members` - ワークスペースメンバー
- `projects` - プロジェクト
- `tasks` - タスク

ENUM 型:
- `workspace_role` - owner, admin, member, guest
- `task_status` - todo, in_progress, done, cancelled
- `task_priority` - low, medium, high, urgent

インデックス:
- `ix_users_email` - ユーザーメール
- `ix_workspaces_slug` - ワークスペース slug
- `ix_workspaces_owner_id` - 所有者 ID
- `ix_workspace_members_workspace_id` - ワークスペース ID
- `ix_projects_workspace_id` - ワークスペース ID
- `ix_tasks_project_id` - プロジェクト ID
- `ix_tasks_assignee_id` - 担当者 ID
- `ix_tasks_status` - ステータス
- `ix_tasks_due_date` - 期限

4. 動作確認

```
$ alembic upgrade head
INFO  [alembic.runtime.migration] Context impl PostgresImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.
INFO  [alembic.runtime.migration] Running upgrade  -> 0001, initial
```

```
$ alembic current
0001 (head)
```

```
$ alembic history
0001 (head), initial
```

```
$ python -c "from app.models import User, Workspace, WorkspaceMember, Project, Task; print('OK')"
OK
```

次のステップ

1. 認証 API（JWT）

参考資料

- Phase 1 タスク一覧
- 開発環境セットアップ

関連ファイル

- `backend/app/models/base.py`
- `backend/app/models/mixins.py`
- `backend/app/models/user.py`
- `backend/app/models/workspace.py`
- `backend/app/models/project.py`
- `backend/app/models/task.py`
- `backend/alembic/versions/0001_initial.py`

認証 API（JWT）

作成日: 2025年10月7日
ステータス: ✅ 完了

概要

認証 API（JWT）の実装が完了しました。アクセストークン（15分）とリフレッシュトークン（7日）を使用した安全な認証を実装しました。

完了内容

1. 認証フロー

2. 実装内容

JWT ユーティリティ:
- `create_access_token` - アクセストークン生成（15分）
- `create_refresh_token` - リフレッシュトークン生成（7日）
- `decode_token` - トークンデコード
- `verify_token_type` - トークンタイプ検証

パスワードハッシュ:
- `hash_password` - bcrypt ハッシュ化
- `verify_password` - パスワード検証

Redis 連携:
- `store_refresh_token` - リフレッシュトークン保存
- `is_refresh_token_valid` - 有効性チェック
- `revoke_refresh_token` - 単一トークン無効化
- `revoke_all_user_tokens` - 全トークン無効化

依存関係:
- `get_current_user` - 認証ユーザー取得
- `get_current_user_optional` - オプショナル認証
- `get_current_active_superuser` - 管理者チェック

エンドポイント:
- `POST /auth/register` - ユーザー登録
- `POST /auth/login` - ログイン
- `POST /auth/refresh` - トークンリフレッシュ
- `POST /auth/logout` - ログアウト
- `GET /auth/me` - 現在のユーザー情報
- `PUT /auth/me` - プロフィール更新
- `PUT /auth/me/password` - パスワード変更
- `GET /auth/protected` - 認証テスト用

3. 動作確認

```
$ curl -X POST http://localhost:8000/auth/register -H "Content-Type: application/json" -d '{"email":"test@example.com","password":"Test1234"}'
{"id":"019...","email":"test@example.com","is_active":true,"is_superuser":false,"created_at":"2025-10-07T...","updated_at":null}
```

```
$ curl -X POST http://localhost:8000/auth/login -H "Content-Type: application/x-www-form-urlencoded" -d "username=test@example.com&password=Test1234"
{"access_token":"eyJ...","refresh_token":"eyJ...","token_type":"bearer"}
```

```
$ curl http://localhost:8000/auth/me -H "Authorization: Bearer eyJ..."
{"id":"019...","email":"test@example.com","is_active":true,"is_superuser":false,"created_at":"2025-10-07T...","updated_at":null}
```

```
$ curl -X POST http://localhost:8000/auth/refresh -H "Content-Type: application/x-www-form-urlencoded" -d "refresh_token=eyJ..."
{"access_token":"eyJ...","refresh_token":"eyJ...","token_type":"bearer"}
```

```
$ curl -X POST http://localhost:8000/auth/logout -H "Content-Type: application/x-www-form-urlencoded" -d "refresh_token=eyJ..."
{"message":"Successfully logged out"}
```

次のステップ

1. ワークスペース API

参考資料

- Phase 1 タスク一覧
- データベースモデル

関連ファイル

- `backend/app/core/security.py`
- `backend/app/schemas/auth.py`
- `backend/app/schemas/user.py`
- `backend/app/services/auth_service.py`
- `backend/app/services/redis_service.py`
- `backend/app/dependencies/auth.py`
- `backend/app/dependencies/__init__.py`
- `backend/app/api/v1/auth.py`
- `backend/app/api/v1/__init__.py`

ワークスペース API

作成日: 2025年10月7日
ステータス: ✅ 完了

概要

ワークスペース API の実装が完了しました。マルチテナント対応のワークスペース管理を実装しました。

完了内容

1. 認可フロー

リクエスト
JWT 検証
get_current_user
get_workspace_member(workspace_id, user, db)
SELECT workspace_members WHERE workspace_id AND user_id
member
member = None
403 Forbidden
role_hierarchy[member.role] >= role_hierarchy[required_role]
true
false
403 Forbidden
ハンドラ実行
owner > admin > member > guest
GET /workspaces/{id}
member
member
member
member
owner
admin
member
guest
PUT /workspaces/{id}
owner
admin
member
guest
DELETE /workspaces/{id}
owner
admin
member
guest
POST /workspaces/{id}/members
owner
admin
member
guest
PUT /workspaces/{id}/members/{user_id}
owner
admin
member
guest
DELETE /workspaces/{id}/members/{user_id}
owner
admin
member
guest
GET /workspaces/{id}/members
owner
admin
member
guest
POST /workspaces/{id}/transfer
owner
admin
member
guest
owner
admin
member
guest
403 Forbidden
owner
admin
member
guest
403 Forbidden
owner
admin
member
guest
403 Forbidden